Efficient Critical Path Identification Based on Viability Analysis Method Considering Process Variations

Sheis Abolmaali, Nika Mansouri-Ghiasi, Mehdi Kamal, Ali Afzali-Kusha, and Massoud Pedram

***Abstract*— In this brief, we propose an effective adaptation of viability analysis in statistical static timing analysis. The adaption benefits well from a dynamic programming implementation of the viability function. For a rapid identification of statistical longest true paths, the technique makes use of a fast preprocessing step identifying the gates with a small probability of being viable in the circuit, and a number of simple optimization techniques. This makes the approach fast without lowering its accuracy. The efficacy of the proposed statistical timing analysis is assessed using ISCAS benchmark circuits and carry skip adders. The results show that the proposed technique leads to, on average, 18× higher speed compared to those of the state-of-the-art technique. This improvement is achieved at the cost of −1.7% precision lost compared to that of the Monte-Carlo method.**

***Index Terms*— Process variation, statistical static timing analysis (SSTA), true critical path, viability analysis.**

I. Introduction

Static timing analysis (STA) is invoked to identify the critical path of circuit in a design cycle without any simulation. Only true critical paths, which can be sensitized by proper input vectors, have impact on the circuit clock frequency [1]. In dynamic path sensitization method, in contrast to the static one, temporal and transitional values on the input pins of each gate in the path are considered in the analysis [1], [2]. In the presence of the variations, criticality and trueness of paths may vary in different instances of the fabricated circuit [3]. Therefore, in statistical STA (SSTA), delays and trueness probabilities of paths are modeled by statistical distributions [3]. In this analysis, instead of one critical path, a set of the most probable longest paths should be considered.

A few works which used SSTA for identifying critical paths contemplate false (not true) paths in their proposed methods (see [2], [4]–[6]). In [4], the dependencies of gate delays to the gate input vectors were analyzed which were joined by identifying the impact of false paths expected to be critical under variations. The results of [5] for ISCAS'89 benchmark circuits indicated that a significant portion of the longest paths were actually false. The algorithm of [6] was based on a branch and bound framework which pruned subpaths that were not true, or their criticality probabilities were low. The run times of these works were long due to unnecessary computational overheads. In addition, many of them were using satisfiability (SAT) solvers and static sensitization method which were imprecise. Also, they did not consider explicitly variations in checking the sensitization of gates. In [2], a viability analysis technique, introduced in [1], which considers statistical gate delay distribution has been proposed. The authors used the equivalence of AND/OR operations, which employed in primary viability analysis to check the dynamic sensitization of a path, and statistical MIN/MAX operations, to perform viability checking on random variables of gate delays. The work in [2] did not use best-first search method which efficiently identify critical paths in the primary viability analysis. Also, statistical MAX/MIN operations, which are computationally expensive and inaccurate in nature, were used frequently in the approach of [2].

In this brief, we propose a fast yet accurate method to identify the most probable longest true paths of the circuit in the presence of the process variation. The method, which uses path-based SSTA with dynamic sensitization, is based on the viability analysis and properly integrates statistical quantities, such as delay distributions, into basic logical expressions of the viability. An efficient implementation of the algorithm is achieved by properly utilizing the dynamic programming implementation of the viability function. The proposed method finds the trueness probability distributions of the longest paths, instead of just finding the trueness probability values.

TABLE I

Viability Formulas for Primary Viability Function [(1)–(3)] and Statistical Viability Function [(4)–(6)]

$\psi_P = \prod_{i=1}^{m} \psi_P^{f_i}$ (1)	$\varphi_P = \min_{i=0..m} \varphi_P^{f_i}$ (4)
$\psi_P^{f_i} = \sum_U ((S_U \frac{\partial f_i}{\partial f_{i-1}}) \prod_{g \in U} \psi^{g,\tau_{i-1}})$ (2)	$\varphi_P^{f_i} = \max_{U \subseteq S(f_i,P)} \left(S_U \frac{\partial f_i}{\partial f_{i-1}} . \min_{g \in U} \varphi^{g,\tau_{i-1}} \right)$ (5)
$\psi^{g,t} = \sum_{P \in \mathcal{P}_{g,t}} \psi_P$ (3)	$\varphi^{g,t} = \max_{Q \in P_g} \left(\min(\varphi_Q, d(Q) - t) \right)$ (6)

II. Viability

One of the well-known dynamic sensitization methods is the viability analysis, where a viability function is utilized for checking the sensitization of each node in the path. The function considers the ability of other nodes in the circuit in sensitizing each node [1]. An input to a node lying on a given path is named online input while the other inputs of the node are named side inputs. The late side input is a side input which has a transition from noncontrolling value (NCV) to controlling value (CV) after the transition on the on-line input. Thus, it temporarily can sensitize the node [2]. To analyze whether a gate can propagate a transition from one of its input pins, for a given input vector v, the viability analysis discovers a number of paths which allow passing of transitions originated from the primary inputs of circuit to the gate inputs. These paths are called viable paths [1]. More specifically, [1] stated that a path P, consisting of the nodes $\{f_0, \ldots, f_m\}$, is viable under an input vector v, iff, for each f_i and each side input $g \neq f_{i-1}$ to f_i, either 1) g is set to its sensitizing value by v or 2) g terminates a viable path under v with a delay longer or equal than the delay of the partial path $\{f_0, \ldots, f_{i-1}\}$.

The viability function of the path P (ψ_P) is defined by the Boolean functions which are expressed by (1)–(3) in Table I [1]. Here, the multiplication (addition) means logical AND (OR). Equation (1) states that a path will be viable if all of its nodes are viable. In (2), all of the cases where the node f_i of P can be viable are included. Also, U is a subset of the side inputs of f_i which have a CV, and S_U denotes the side inputs which are not in U. Each g in U must terminate at

Manuscript received December 3, 2016; revised March 10, 2017 and May 3, 2017; accepted May 4, 2017. Date of publication May 26, 2017; date of current version August 23, 2017. The work of S. Abolmaali, N. Mansouri-Ghiasi, M. Kamal, and A. Afzali-Kusha was supported by the Iran National Science Foundation. *(Corresponding author: Ali Afzali-Kusha.)*

S. Abolmaali, N. Mansouri-Ghiasi, M. Kamal, and A. Afzali-Kusha are with the ECE Department, University of Tehran, Tehran 14399-57131, Iran (e-mail: sh.abolmaali@ut.ac.ir; n.mansorighiasi@alumni.ut.ac.ir; mehdikamal@ut.ac.ir; afzali@ut.ac.ir).

M. Pedram is with the Department of Electrical Engineering, University of Southern California, Los Angeles, CA 90089-2562 USA (e-mail: pedram@usc.edu).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TVLSI.2017.2703623

1063-8210 © 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:02:56 UTC from IEEE Xplore. Restrictions apply.

least one longer viable path to make f_i sensitizable. This has been formulated in (3) as $\psi^{g,t}$. Note that $P_{g,t}$ shows the set of paths terminating on g with a delay value longer or equal than t. Also, $S_U(\partial f_i/\partial f_{i-1})$ denotes that the other side inputs out of U have a sensitizing value, and hence, are not considered in the node viability calculation. Finally, the Σ operation over U (Σ_U) indicates that all the possible subsets of f_i's side inputs should be evaluated. The path P is viable under input vector v iff $\psi_P(v) = 1$.

The best-first procedure, as a common search method for finding the longest path [7], maintains the partial true paths in a priority queue ordered by the potential full length, or the length of the longest extension of the partial path. This quantity (i.e., potential full length) is named the esperance (ESP) of the partial path. The search in the best-first procedure is terminated when an output is reached. This is due to the fact that if such a longer path exists, it should be identified sooner in the best-first search process. The best-first search generally examines fewer paths in comparison to the depth-first search [7]. The most reasonable quantity that may be chosen for expressing the ESP (length of the longest extension) of a partial path is the required time (RT) of nodes. The RT is defined as the maximum time needed for a transition to travel from the node to a primary output. If a node has a longer RT, there will be a higher probability of being critical for the paths passing through this node. The RTs, which are obtained by performing conventional block-based SSTA on the circuit, are pessimistically inaccurate. We, however, use it as a first step to provide a set of candidate paths which are processed further to determine if they can be considered as critical viable paths.

The viability of a path depends on all the adjoining paths having equal or longer lengths compared to this path length [1]. It is required that the viability function be computed for each side input to each encountered node on the (candidate) path. In the best-first search method, before evaluating the candidate path, every side path, longer than the candidate path, has been traced. This feature of the method enables us to avoid using a time-consuming recursive approach for finding longer side viable paths. The best-first search method may be implemented using the dynamic programming approach explained in detail in [1].

III. Viability Analysis Considering Process Variation

Under the process variation, gate delays have statistical behaviors being usually expressed by random variables. In this brief, to model gate delays we use the first order canonical model of [8], and to properly include spatial correlation of process parameters the grid model of [9] is utilized. Here, to determine the relative greatness of the path lengths, one may use the tightness probability (TP). For two random variables X and Y, the TP of X is the probability of $X > Y$. For comparing two random variables, we use the TP formulas in [10], which is also used in [8], for computing the MIN/MAX operations.

Jung and Kim [2] proposed an approach (which is used in this brief) for converting Boolean formulas (1)–(3) into a form which considers the statistical nature of the quantities under process variation. They utilized the theorem which states that logical AND/OR operations and statistical MIN/MAX operations are equivalent when it is needed to know that the probability of a random variable being larger than zero. This equivalence results in (4)–(6) are given in Table I. The viability (probability of a path to be viable) of the path P under an input vector γ is defined as Prob$[\varphi_P(\gamma) > 0]$ [2]. Here, $\varphi_P(\gamma)$ is the viability distribution of P under γ.

IV. Proposed Improvement Techniques

A. Simplification in Viability Formulas Under a Test Vector

The Σ operations in (2) and (3) imply that there might be several combinations of logical variable values satisfying the viability conditions where one or more input vectors might exist satisfying the conditions. Hence, the Σ operations are used to find different possible values for the input vectors. This is not the case when evaluating the viability condition for a given test vector. In this case, for a node f_i in the path, it is determined which one of its inputs have NCV, and hence, there is no need to consider all the possible combinations of the side inputs. For example, for the gate shown in Fig. 1(a), inputs 1 and 2 of g are in U, and inputs 3 and 4 are in S_U. Therefore, the Σ operation in (2) on all the subsets of U can be removed and the equation simplified as $\prod_{g\in U} \psi^{g,\tau_{i-1}}$. Also, since the value of $S_U(\partial f_i/\partial f_{i-1})$ in (2) is logical 1, it is omitted from this equation. Thus, (5) can also be simplified as $\min_{g\in U}\varphi^{g,\tau_{i-1}}$. Removing the MAX operations reduces the computational load of evaluating the viability of a path node.

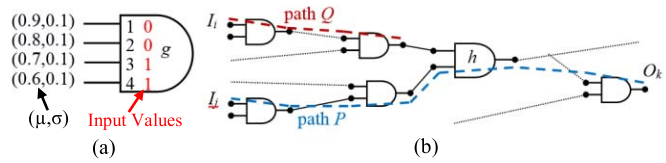

Fig. 1. (a) Example gate with value and the corresponding mean and standard deviation of the AT for each input. (b) Circuit example for illustrating the relation between viability and delay distributions of the longest viable path.

B. Proposed Viability Function Under Delay Variations

As in the primary viability analysis, the RT of the node is used as ESP. The approach to reduce the required computations for analyzing the viability of a node is explained as follows. Since every viability quantity becomes probabilistic, at each terminating node of a partial path, we just determine if this node has large enough viability (can be viable). The exact probability value of the viability is not important here. The reason is that we wish to assess that if this partial path can terminate on an output node yielding a path with the longest delay distribution. If a node does not have the required conditions, it is omitted from further computations.

To assess the viability of a candidate fan-out node g to extend a partial path, each side input k to g must satisfy one of the following conditions: First, k must have a sensitizing value (the simplest condition). Otherwise, at least one longer viable path must terminate on k. Since in the proposed viability function, the exact computations needed for obtaining the viability distribution of the node is avoided, the list of such paths for each side input k of g is held. Therefore, instead of maintaining the exact distributions of the viabilities, knowing the number of longer side paths is enough here. If the number of such paths is 0 for a side input, it is still not certain that the node cannot be sensitized. The reason is that the strategy in the dynamic implementation is to trace partial paths with higher ESPs. This means that there might be viable side paths having greater delay distributions with lower probabilities. These side paths have not been encountered until now due to the employed best-first search strategy.

To consider the effect of such paths, instead of tracing back all side paths toward the input nodes of circuit, we use a heuristic method where the arrival times (ATs) of nodes are considered as a metric. The AT of the node is defined as the delay of the longest partial path from the primary inputs of the circuit into the node. For the node g, if the side input k has a sensitizing value or its list of terminating longer viable paths is not empty, k can sensitize the path passing through the online input f of g. Otherwise, the viability function compares (by calculating TP) the AT of k and the delay of the candidate partial path ending onto f. If the AT is greater than the delay of the partial path,

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:02:56 UTC from IEEE Xplore. Restrictions apply.

this means that k can have terminating long viable paths with a high probability, which have not been traced yet. Hence, it is assumed that k can satisfy the viability conditions, and thus, should not be rejected. The partial path terminating on g through f denoted by P_{fg}, is added to the pending list of the paths having the chance of becoming viable. After examining some new partial paths, there might be a new viable partial path terminating onto g through k. If this happens, the list of terminating longer viable paths of k is not empty anymore and P_{fg} becomes really a viable path, although probably with a lower viability. It comes out from the pending list and is immediately evaluated (due to its larger ESP). The process, which has a small timing overhead, needs to be run for such pending paths. There are not many of these paths.

One last important point is that in examining the viability of each node of the path in our viability function, none of the MAX/MIN operations in (4)–(6) is performed. The reason is that, in the first step, the possibility of a node being viable is explored using TP. The MIN/MAX operations require the calculation of the sensitivity coefficients to the physical variational parameters making the operations computationally more expensive than TP.

C. Improvement in ESP Factor

To improve the quality of the ESP factor, we propose a pruning method. The good selection of ESP reduces the number of paths to be evaluated. The pruning method also uses ATs in a preprocessing step before beginning the main procedure for finding the longest viable paths. For every node of a circuit, a counter is defined for each of its inputs. If an input has a NCV, then the counters of other inputs are incremented, indicating that this input has the required condition to sensitize them. On the other hand, if it has a CV, its AT is compared to the ATs of each of the other inputs. If the AT of this input is greater than that of the other ones, this node has a high chance of acting as a late side input sensitizing the other inputs. Hence, the counters of the other inputs should be incremented. Then, during finding the RTs of the nodes, if an input counter of a circuit node has a value less than the number of node inputs minus one, it will indicate that at least one of its side inputs does not satisfy the viability conditions. Thus, it is pruned from the RT calculation. This way, the ESP directs the partial paths to the nodes with higher viability probabilities.

The pruning method may be explained better using the four-input AND gate g of Fig. 1(a). The initial values of all the counters ($\text{cnt}_1, \ldots, \text{cnt}_4$) are considered to be 0. Since the inputs 3 and 4 have the sensitizing value, the counters become (2, 2, 1, 1) [input 3 $\rightarrow$ (1, 1, 0, 1); input 4 $\rightarrow$ (1 + 1, 1 + 1, 0 + 1, 1 + 0)]. Since the input 1 has the CV and its AT has the greatest distribution using the TP among the distributions of other ATs, it could be act as a late side input for the other inputs. Thus, the counters are updated to (2, 3, 2, 2). Input 2 also has the CV and its AT is greater than those for inputs 3 and 4. Thus, at the end, the counter values become (2, 3, 3, 3). Since cnt_2, cnt_3, and cnt_4 have values equal to the number of the inputs of g minus one, inputs 2 through 4 are not pruned, while input 1 is pruned. Note that if all the inputs of the gate g have the sensitizing value, no node will be pruned. On the other hand, if all the inputs have the CV, only input 4, which has the smallest AT, can be sensitized.

D. Adjustment Parameter for Viability Distribution

It is imperative to state that the longest path (with the maximum delay) does not necessarily have the highest viability distribution (and vice versa) as may be explained using the circuit of Fig. 1(b). Consider the path P containing N nodes, where $N - 1$ nodes are statically sensitized. Also, assume that the remaining node, say, h, which can be anywhere in the path, is a two-input gate. It has exactly one viable path terminating on its side input with the viability of 1. (All the nodes of the side path are also statically sensitizable.) Using (4)–(6), one finds the viability of the path equal to $d(Q) - t$ where $d(Q)$ is the delay distribution of the side path and t denotes the delay distribution of the partial path ending at the node on the online input. Therefore, the viability distribution becomes larger when $d(Q)$ becomes greater than t. In general, there is no specific dependence between $d(Q)$ and t, and the delay of the entire path. Hence, if we assume that P is the longest viable path, and $d(Q)$ and t have about the same distributions, it is possible for the longest viable path to have a viability distribution with a very small mean value. The opposite case is also possible where a shorter path has a greater viability distribution if there is a large difference between

Algorithm 1 Overall Procedure to Find K Longest Viable Paths Under Process Variations

```
L is the list of longest viable paths
PL is the list of pending probable viable paths

function ViabilityFunction(Path p)
1.    g is the last node of p, and f is its online input node
2.    res = true;
3.    for each node k ∈ side inputs of g do
3.1.        if k has non-controlling value then continue;
3.2.        if # of side viable paths terminating on k is > 0 then continue;
3.3.        if TP of AT distribution of k, relative to delay distribution of
            the partial path of p ending in f, is greater than 0.5 then
3.3.1.          add a copy of p into PL
3.4.        res = false; break;
4.    return res;

function FindKLongestTruePaths(Circuit C, input vector v)
      p is a partial path, and g is its last node
      …
      psi = ViabilityFunction(p)
      if psi == true then
          if g is a primary output node then insert p into L
      …

function OverallProcedure(Circuit C)
1.    find a set of input vectors V which can sensitize top longest viable
      paths of C
2.    for each vector v ∈ V do
2.1.        perform a simple forward SSTA to determine AT distributions
            of all nodes in C, along with determining static values of nodes,
            along with the pruning procedure
2.2.        perform a simple backward SSTA to find RT distributions of
            all nodes in C
2.3.        FindKLongestTruePaths(C, v)
3.    select K paths with greatest delay distributions from L, as the final
      result of algorithm
```

TABLE II
STATISTICAL PARAMETERS OF THE LONGEST VIABLE PATH DELAYS PRODUCED BY THE ALGORITHMS

Circuit	MC		This Work		[2]		SSTA	
	μ	σ	μ	σ	μ	σ	μ	σ
s15850	1.64	0.15	1.64	0.14	1.64	0.14	1.70	0.17
s35932	0.57	0.06	0.57	0.06	0.57	0.06	0.57	0.08
s38417	1.02	0.13	1.02	0.13	1.02	0.13	1.22	0.15
s38584	0.97	0.11	0.97	0.11	0.97	0.11	0.99	0.13
CSKA 16×2	1.10	0.09	1.10	0.08	1.10	0.08	1.67	0.10
CSKA 16×4	1.35	0.10	1.35	0.10	1.35	0.10	1.69	0.11
CSKA 32×2	2.19	0.15	2.19	0.15	2.19	0.15	3.42	0.18
CSKA 32×4	2.85	0.18	2.85	0.18	2.85	0.18	3.46	0.21
Avg. error with MC (%)			0.02	-1.7	0.02	-1.7	22.6	17.0

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:02:56 UTC from IEEE Xplore. Restrictions apply.

TABLE III

RESULTS FOR AVERAGE ERROR OF THE VIABILITY VALUES, STATISTICAL PARAMETERS OF THE LONGEST VIABLE PATH DELAYS, REQUIRED CPU TIMES FOR THE ALGORITHMS, AND NUMBER OF PROCESSED NODES WITH VP LESS THAN 0.9

Circuit	# of Gates	Average Error of Viability Values (%)	MC (s)	Our Runtime (s)	Runtime of [2] (s)	SSTA RT/AT (s)	Our Runtime/ Runtime of [2]	# of Processed Nodes with VP < 0.9				Total # of Processed Nodes		
								With Pruning		Without Pruning				
								#	% to # of Gates	#	% to # of Gates	This Work	[2]	Ratio (%)
s15850	3261	0.09	41715	8.47	99.28	1.51/0.34	11.7	2	0.06	172	5.27	498	3261	15.27
s35932	8525	0.04	25241	3.12	128.60	2.39/0.38	41.2	21	0.25	1446	16.96	358	8525	4.20
s38417	9779	0.01	34469	5.72	288.49	3.97/0.58	50.4	0	0	241	2.46	157	9779	1.61
s38584	10793	0.04	12816	6.63	201.53	4.89/0.46	30.4	0	0	204	1.89	262	10793	2.43
CSKA 16×2	210	0.40	5396	2.14	8.7	0.75/0.28	4.1	3	1.43	170	80.95	257	210	122.38
CSKA 16×4	226	0.30	9897	3.64	10.78	0.87/0.54	2.9	8	3.54	293	129.65	305	226	134.95
CSKA 32×2	418	0.70	18583	13.45	28.69	0.21/0.10	2.1	8	1.91	539	128.95	480	418	114.83
CSKA 32×4	458	0.70	71052	35.72	51.86	0.25/0.25	1.4	8	1.75	621	135.59	631	458	137.77
Average	4208	0.285	27396	9.86	102.24	1.85/0.37	18.0	6	1.12	461	62.71	368	4209	66.68

$d(Q)$ and t distributions. In this brief, we consider a user-defined parameter, φ_{th}, $(0 \leq \varphi_{\text{th}} \leq 1)$ which determines the minimum viability probability (VP) of the longest path.

E. Overall Algorithm

Algorithm 1 shows the overall procedure for finding the K longest viable paths under the process variation. The function viability function is explained in Section ýIV-B and is used in the function FindKLongestTruePaths. The latter function is the implementation of the dynamic programming procedure for finding the K longest viable paths (which is path-based) as explained in [1]. It considers the delay variation. The overall procedure begins with extracting a set of useful input vectors providing the conditions for sensitizing the top longest paths. Here, the technique proposed in [2], which efficiently sensitized the longest viable paths in the presence of variation, has been adopted. Then, for each extracted vector, a simple forward SSTA analysis is performed from the input nodes of the circuit to find the AT distributions for each node. Finding ATs is accompanied by finding the static values of the circuit nodes along with performing the pruning. Then, a simple backward SSTA from the output nodes of the circuit is executed to find the RT distributions of all the nodes obtaining their ESPs. The function FindKLongestTruePaths is called after taking the above steps. This function finds the K longest viable paths of the circuit in the presence of the delay variation and stores them in a related list descending. For several input vectors, this procedure is performed and at the end, the K top longest viable paths in the list are chosen as the final result of the algorithm.

V. RESULTS AND DISCUSSION

In this brief, six variational process parameters of length, width, and threshold voltage for nMOS and pMOS transistors with the maximum variation of (σ/μ) of 15% were considered. Sequential ISCAS benchmarks as well as a number of carry skip adder (CSKA) circuits were used as test circuits. For each circuit, a maximum number of 20 critical paths (K) were found. For extracting the good input vectors in the primary viability analysis, we used the SAT solver of [11]. We implemented our algorithm in C++ and ran it on a Linux machine with an Intel Core i7 CPU (4790 at 3.60 GHz) and 8-GB RAM. Gate delays and sensitivities of the delay quantities to parameter variations were obtained for each gate input separately. Here, SPICE simulations were used with a SPICE library of the NanGate45 package [12]. Since our study was focused on assessing the efficacy of the methods, all the results presented here are for runs under one input vector.

In Table II, the statistical parameters for the distributions of the longest viable paths obtained from different methods of the proposed work, Monte Carlo (MC) [2], and a basic SSTA are compared. The results reveal a very good accuracy of the proposed work compared to the MC simulation approach. Table III, which contains several group of results, is partitioned into five regions distinguished by white and gray background colors. The first region contains the name and number of gates for each test circuit while the second region shows the average error of the viability values of top 20 true longest paths extracted by the proposed algorithm when compared path-by-path to the MC approach. For the MC approach, the primary viability was used for 10 000 instances of each test circuit. The results indicate a high accuracy for the proposed approach with errors no more than 0.7%. In the third region, we provide the CPU time needed for finding the top 20 longest paths, excluding the run-times for input vector extraction. The run times related to our work include two SSTA steps needed for ATs and RTs computation. The two numbers in the SSTA column denote the run times for these steps, respectively. For the ISCAS circuits, it is observed that the time required for calculating these quantities can form a major part of the execution time. Our method requires much smaller CPU time relative to that of [2]. The advantage of our approach is higher for the ISCAS circuits where, for example, for the circuit s38417, a ratio of 50.4× was achieved.

The fourth region compares the number of the nodes which have a small probability of being viable and are processed during the longest path search of the circuits. They are for two cases of using and not using the pruning method. Here, we consider $\varphi_{\text{th}} = 0.9$, and hence, the number of the nodes with a VP value of less than 0.9 is reported. The numbers in the first column of the region are the total number of nodes with VP less than 0.9 for each circuit. The third column of the region provides the same information for the cases without the pruning method. It is evident that a great reduction in the number of processed nodes may be achieved. For the circuits s38417 and s38584, no low viable node was considered. Indeed, all of these nodes have been removed (before search process for determining the longest viable paths) by employing our proposed pruning method. Furthermore, a 86× reduction in the number of processed low viable nodes was obtained in the case of s15850. Note that the reduction of even one low viable node can have a considerable impact on the runtime, by not processing long partial paths containing this low viable node.

The last region of Table III compares the numbers of the nodes processed during the execution of the proposed algorithm and those of [2]. For the algorithm of [2], because of its block-based timing analysis nature, all the nodes of the circuits were processed. In our

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:02:56 UTC from IEEE Xplore. Restrictions apply.

work, which is path-based, the ESP factor in the best-first search method guides the partial paths toward the longest viable paths, which leads to ignoring many circuit nodes in the viability analysis. The results demonstrate the higher efficiency of our path-based approach for the circuits with large number of gates. More than 11× reduction in the number of processed nodes relative to those of work [2] were obtained.

For the CSKA circuits, the proposed method does not yield very large speedups. It originates from the difference between the ratios of the numbers of the nodes in the critical paths to the total circuit node numbers in the two benchmark sets. For example, for the circuit s38584, the first longest path includes 32 circuit nodes which is a very small fraction of 10 793 circuit nodes. For the CSKA 32×4 circuit, however, the first longest path contains 92 nodes of the circuit 458 nodes which is very large. Therefore, for the CSKA circuits, it is very probable that finding 20 critical paths needs more nodes to be processed than the circuit nodes.

VI. Conclusion

In this brief, we proposed a fast yet accurate timing analysis method under process variations based on the viability analysis. The method adopted statistical delay distributions in the dynamic programming implementation of the primary viability analysis. Specifically, the best-first search method was used properly to reduce the number of the circuit nodes which were processed. Statistical computations of viability function were reduced by simplifying the viability formulas under a given input vector. In addition, the AT of the node was used as a heuristic for the computation of the node sensitivity. By using a pruning algorithm which also utilized the ATs of nodes, the number of the nodes processed during the longest path search was decreased significantly. The comparative results indicated speed improvement and high accuracy for the proposed algorithm.

References

[1] P. C. McGeer and R. K. Brayton, "Efficient algorithms for computing the longest viable path in a combinational network," in *Proc. DAC*, Jun. 1989, pp. 561–567.

[2] J. Jung and T. Kim, "Statistical viability analysis for detecting false paths under delay variation," *IEEE Trans. Comput.-Aided Design Integr.*, vol. 32, no. 1, pp. 111–123, Jan. 2013.

[3] D. Blaauw, K. Chopra, A. Srivastava, and L. Scheffer, "Statistical timing analysis: From basic principles to state of the art," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 4, pp. 589–607, Apr. 2008.

[4] L. Xie *et al.*, "False path aware timing yield estimation under variability," in *Proc. VTS*, May 2009, pp. 161–166.

[5] S.-J. Wang, T.-H. Tzeng, and K. S.-M. Li, "Fast and accurate statistical static timing analysis," in *Proc. ISCAS*, Jun. 2014, pp. 2555–2558.

[6] J. Chung, J. Xiong, V. Zolotov, and J. A. Abraham, "Testability-driven statistical path selection," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 31, no. 8, pp. 1275–1287, Aug. 2012.

[7] J. Benkoski *et al.*, "Efficient algorithms for solving the false path problem in timing verification," in *Proc. ICCAD*, 1987, pp. 44–47.

[8] H. Chang and S. S. Sapatnekar, "Statistical timing analysis considering spatial correlations using a single pert-like traversal," in *Proc. ICCAD*, Nov. 2003, pp. 621–625.

[9] A. Agarwal, D. Blaauw, and V. Zolotov, "Statistical timing analysis for intra-die process variations with spatial correlations," in *Proc. ICCAD*, Nov. 2003, pp. 900–907.

[10] C. E. Clark, "The greatest of a finite set of random variables," *Oper. Res.*, vol. 9, no. 2, pp. 145–162, Mar./Apr. 1961.

[11] N. Eén and N. Sörensson, "An extensible SAT-solver," in *Proc. SAT*, May 2003, pp. 502–518.

[12] *Nangate 45nm Open Cell Library*, accessed on 2010. [Online]. Available: http://www.nangate.com/

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on March 23,2026 at 17:02:56 UTC from IEEE Xplore. Restrictions apply.